# Julia Sets with SIMD

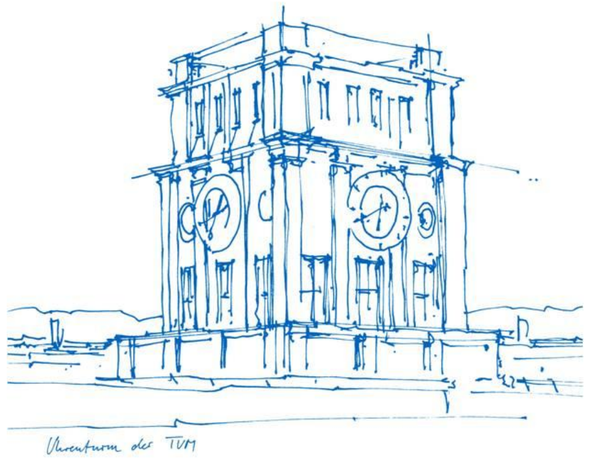
## Implementation, Optimization and Performance Analysis in C

Technical University of Munich

IN0005 Grundlagenpraktikum: Rechnerarchitektur

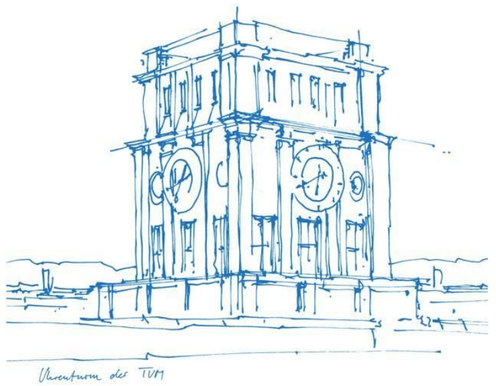
Chair of Computer Architecture and Parallel Systems

**Erdeniz Taşlıçukur · Arda Taş · Noah Schneider**



# Agenda

- Problem Statement and Mathematical Foundation
- Scalar Reference and Baseline SIMD Design
- Correctness Validation and Edge Cases
- Benchmarking methodology and Environment
- Optimization Stages: Measurements and Trade-offs
- Final Performance Summary and Outlook



# Problem Statement and Mathematical Foundation

## 1 Introduction

$z_0 = p,\ z_{i+1} = z_i^2 + c\ (i \geq 0,\ c,\ p \in \mathbb{C})$ · let $z_i = a + bi,\ c = x + yi$

# Problem Statement and Mathematical Foundation

## 1 Introduction

$z_0 = p,\ z_{i+1} = z_i^2 + c\ (i \geq 0,\ c,\ p \in \mathbb{C})$ · let $z_i = a + bi,\ c = x + yi$

$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$

# Problem Statement and Mathematical Foundation

## 1 Introduction

$z_0 = p$, $z_{i+1} = z_i^2 + c$ $(i \geq 0, c, p \in \mathbb{C})$ · let $z_i = a + bi$, $c = x + yi$

$$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$$

**Real part:** $a^2 - b^2 + x$ **Imaginary part:** $2ab + y$

# Problem Statement and Mathematical Foundation

## 1 Introduction

$z_0 = p$, $z_{i+1} = z_i^2 + c$ $(i \geq 0, c, p \in \mathbb{C})$ · let $z_i = a + bi$, $c = x + yi$

$$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$$

**Real part:** $a^2 - b^2 + x$ **Imaginary part:** $2ab + y$

## 2 Iteration Update

The real and imaginary parts become their own update step:

```
vec_next.a = a² − b² + x; vec_next.b = 2ab + y
```

# Problem Statement and Mathematical Foundation

## 1 Introduction

$z_0 = p$, $z_{i+1} = z_i^2 + c$ $(i \geq 0, c, p \in \mathbb{C})$ · let $z_i = a + bi$, $c = x + yi$

$z_{i+1} = (a + bi)^2 + (x + yi) = a^2 + 2abi + (bi)^2 + (x + yi) = a^2 + 2abi - b^2 + (x + yi)$

**Real part:** $a^2 - b^2 + x$ **Imaginary part:** $2ab + y$

## 2 Iteration Update

The real and imaginary parts become their own update step:

```
vec_next.a = a² − b² + x; vec_next.b = 2ab + y
```

## 3 Escape Condition

A point escapes when $|z_i| > 2$, i.e. $\sqrt{(a^2+b^2)} > 2$ for a $|c| \leq 2$. To avoid the square root calculation, we check the squared magnitude instead:

```
magnitude_check = (vec_next.a)² + (vec_next.b)² > 4
```

# Scalar Reference and Baseline SIMD Design

V1

## Scalar Reference: julia_v1()

Pixels are processed one at a time by julia_iterations().

V0

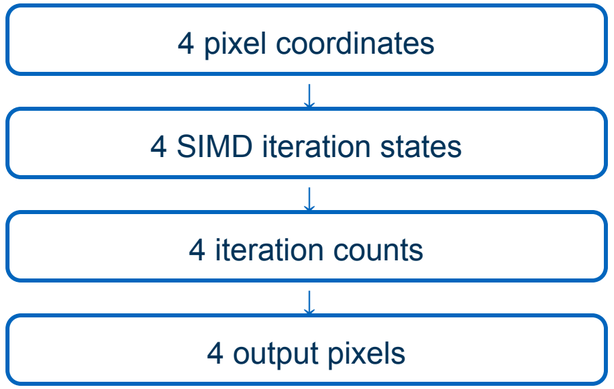
# V0: Four-Pixel SIMD Approach

Four neighbouring pixels are processed using one sequence of vector instructions.

Example: __m128 z_real

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| $zr_0$ | $zr_1$ | $zr_2$ | $zr_3$ |
| Pixel x | Pixel x+1 | Pixel x+2 | Pixel x+3 |
| 32-bit float | 32-bit float | 32-bit float | 32-bit float |

z_imag, counts and active use separate registers with the same lane-to-pixel mapping.




**The same arithmetic instruction operates on all four lanes simultaneously.**

# SIMD Update Step

## Input: four complex states

__m128 z_real

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| $zr_0$ | $zr_1$ | $zr_2$ | $zr_3$ |
| Pixel x | Pixel x+1 | Pixel x+2 | Pixel x+3 |

__m128 z_imag

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| $zi_0$ | $zi_1$ | $zi_2$ | $zi_3$ |
| Pixel x | Pixel x+1 | Pixel x+2 | Pixel x+3 |

↓ both input registers feed both calculation branches ↓

## One lane-wise Julia update

### Real component

real_squared = z_real * z_real

imag_squared = z_imag * z_imag

next_real = real_squared − imag_squared + c_real

Broadcast constant: c_real = [cr, cr, cr, cr]

_mm_mul_ps _mm_sub_ps _mm_add_ps

### Imaginary component

real_imag = z_real * z_imag

next_imag = 2 x real_imag + c_imag

Broadcast constant: c_imag = [ci, ci, ci, ci]

_mm_mul_ps _mm_add_ps

↓ ↓

## Output: four updated complex states

__m128 next_real

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| $zr\_next_0$ | $zr\_next_1$ | $zr\_next_2$ | $zr\_next_3$ |

__m128 next_imag

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| $zi\_next_0$ | $zi\_next_1$ | $zi\_next_2$ | $zi\_next_3$ |

# SIMD Lane Divergence

128-bit Magnitude register (__m128)

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| Pixel (x, y) | Pixel (x+1, y) | Pixel (x+2, y) | Pixel (x+3, y) |

| **SIMD state** | **Lane 0** | **Lane 1** | **Lane 2** | **Lane 3** |
|---|---|---|---|---|
| Previous active state | 0xFFFFFFFF | 0xFFFFFFFF | 0x00000000 | 0xFFFFFFFF |
| $zr^2 + zi^2$ | 3.2 | 4.6 | 1.9 | 5.1 |
| inside (≤ 4) | 0xFFFFFFFF | 0x00000000 | 0xFFFFFFFF | 0x00000000 |
| **New active state** | 0xFFFFFFFF | 0x00000000 | 0x00000000 | 0x00000000 |
| Iteration count | 8 | 7 | 4 | 7 |

This also shows why our active state must be **persistent** : If a lane overflows back into the radius, it must remain.

> **Tail case:** the remaining width % 4 pixels use the scalar path.

```
inside = _mm_cmple_ps(magnitude_squared, four_v);
active = _mm_and_ps(active, inside);

if (_mm_movemask_ps(active) == 0)
    break;
```

inside: result of the current escape test.

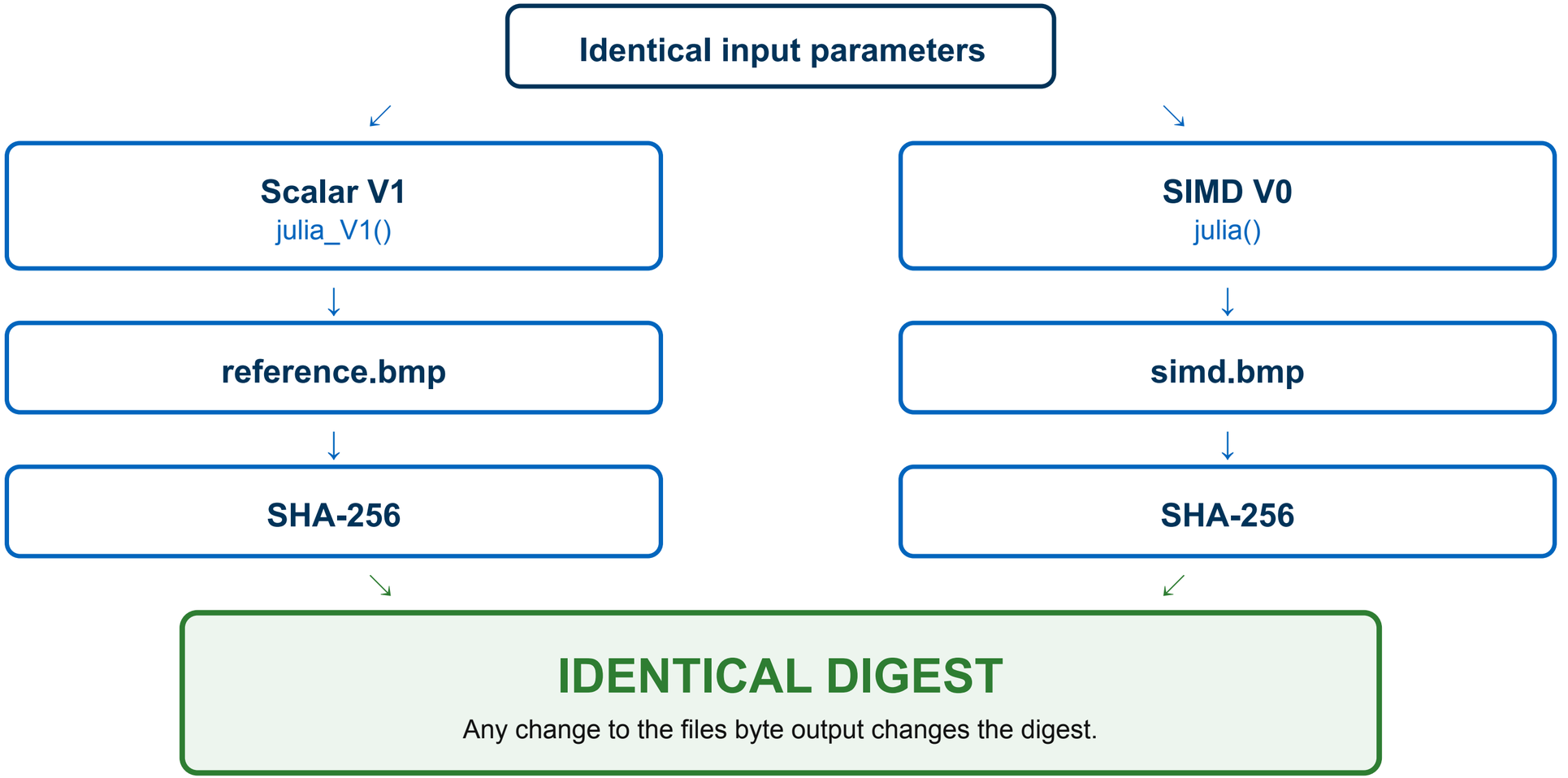
active: persistent state across all iterations.

0xFFFFFFFF: active lane, all bits set.

0x00000000: inactive lane.

movemask: extracts most significant bit per lane; zero means all pixels escaped.

# Correctness




Same iteration

$z_{i+1} = z_i^2 + c$

Same escape condition

$|z_i|^2 \leq 4$

Same iteration limit

$n$

Same pixel conversion

grayscale and color

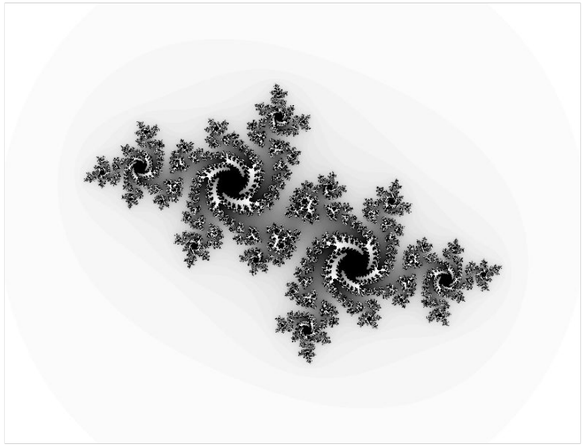
# Correctness

## 7 / 7 matched

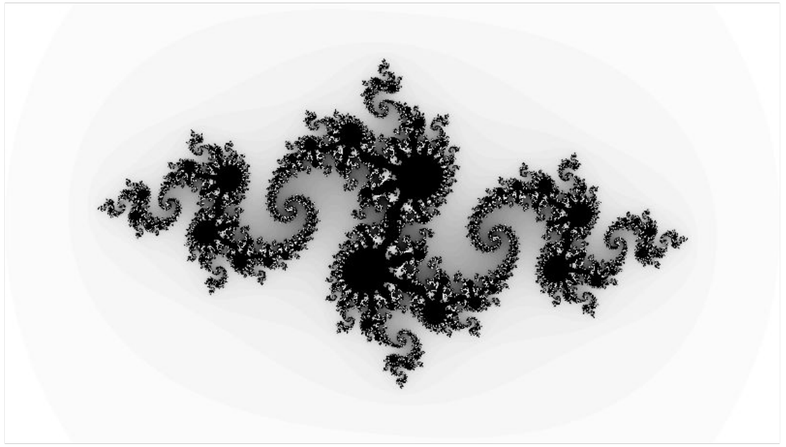
0 mismatches

- ✓ **Default grayscale** Standard Julia calculation
- ✓ **Default color** Vectorized color conversion
- ✓ **SIMD tail, width 801** Final pixel processed scalar
- ✓ **High iteration limit** Correct termination at n
- ✓ **Escaped lanes, c = 1 + 1i** Lanes stop at different iterations
- ✓ **Positive BMP height** Bottom-up row order
- ✓ **n = 0** Loop skipped; black output

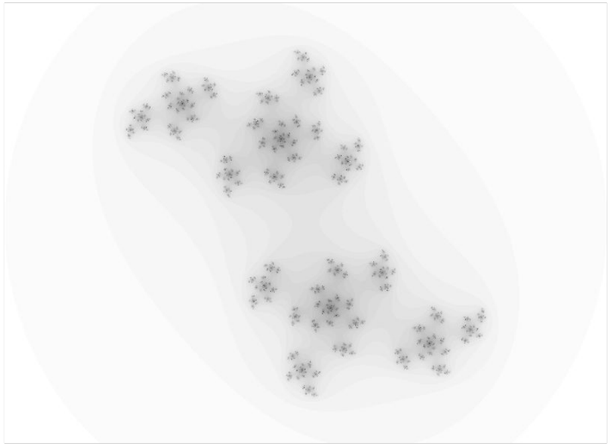
# Outputs Produced by Different c Values



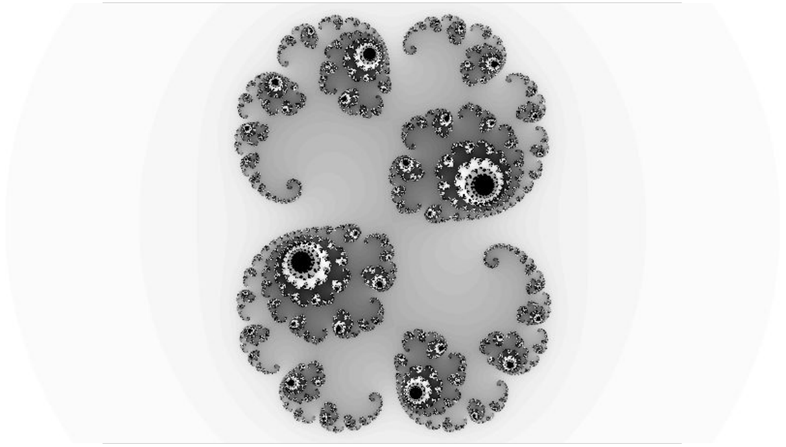
*c = −0.5125 + 0.5213i*



*c = −0.8 + 0.156i*



*c = 0.123 + 0.745i*

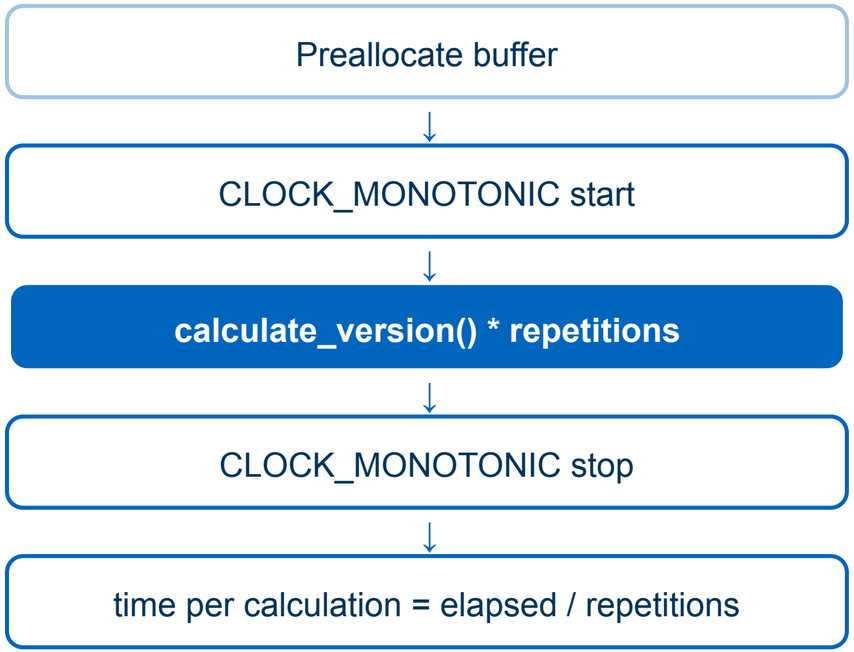


*c = 0.285 + 0.01i*

800 x 600; n = 100; res = 0.005; start = −2 + 1.5i

# Benchmarking Methodology and Environment

All numbers on the following slides were measured the same way, on the same machine.




BMP writing excluded

## Environment

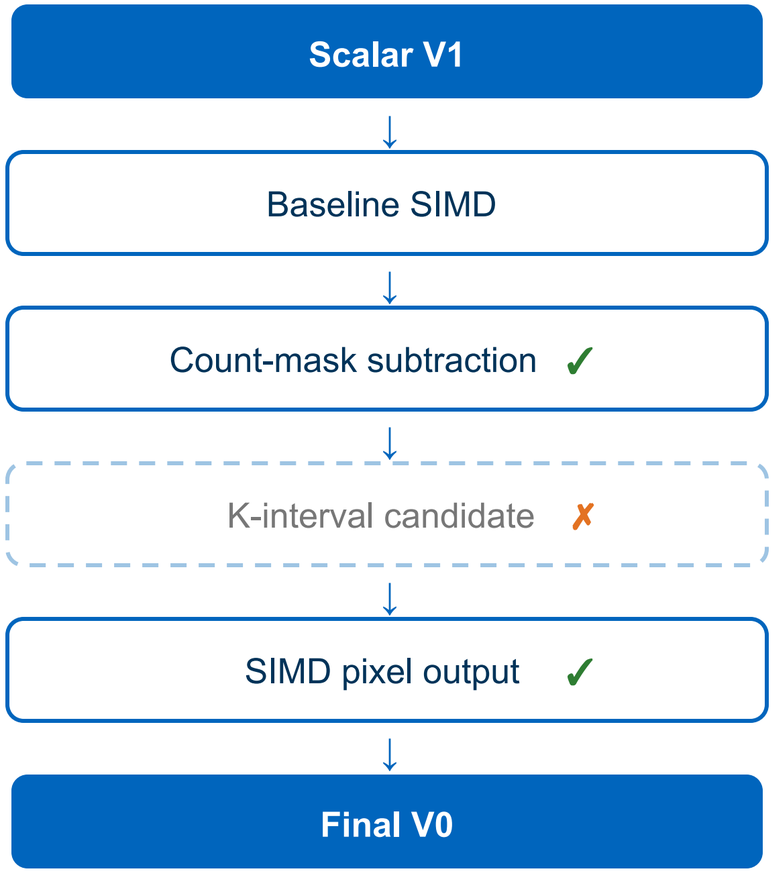
AMD Ryzen 9 5950X - x86-64

GCC 14.2; C17

-O3 -msse4.2

Debian 13.5 under WSL2

# Optimization Strategy



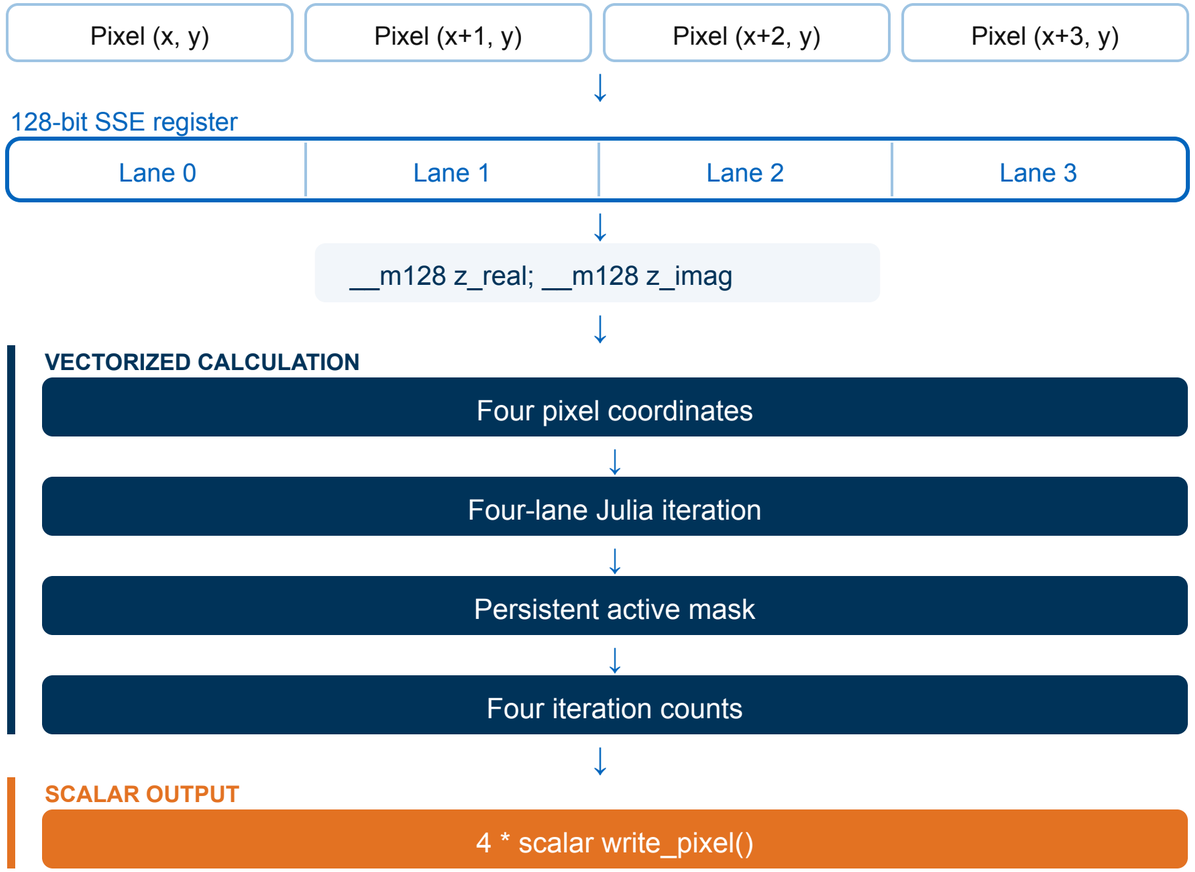

## Chronology

How our final SIMD version developed throughout the project phase.

## Methodology

Optimization targets were selected using profiling, not only theoretical expectations and verified for correctness and benchmarked against the previous version.

# Baseline SIMD




## Initial SIMD Result


# 3.01x speedup

Approximately 75% of the theoretical four-lane SIMD limit

800 x 600 pixels - Grayscale - n = 100 - Average of 100 runs

# Counter Update: AND Then ADD

The initial SIMD implementation first had to convert the active mask to an increment vector.

## Input registers

__m128 active

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| 0xFFFFFFFF active | 0x00000000 inactive | 0xFFFFFFFF active | 0x00000000 inactive |

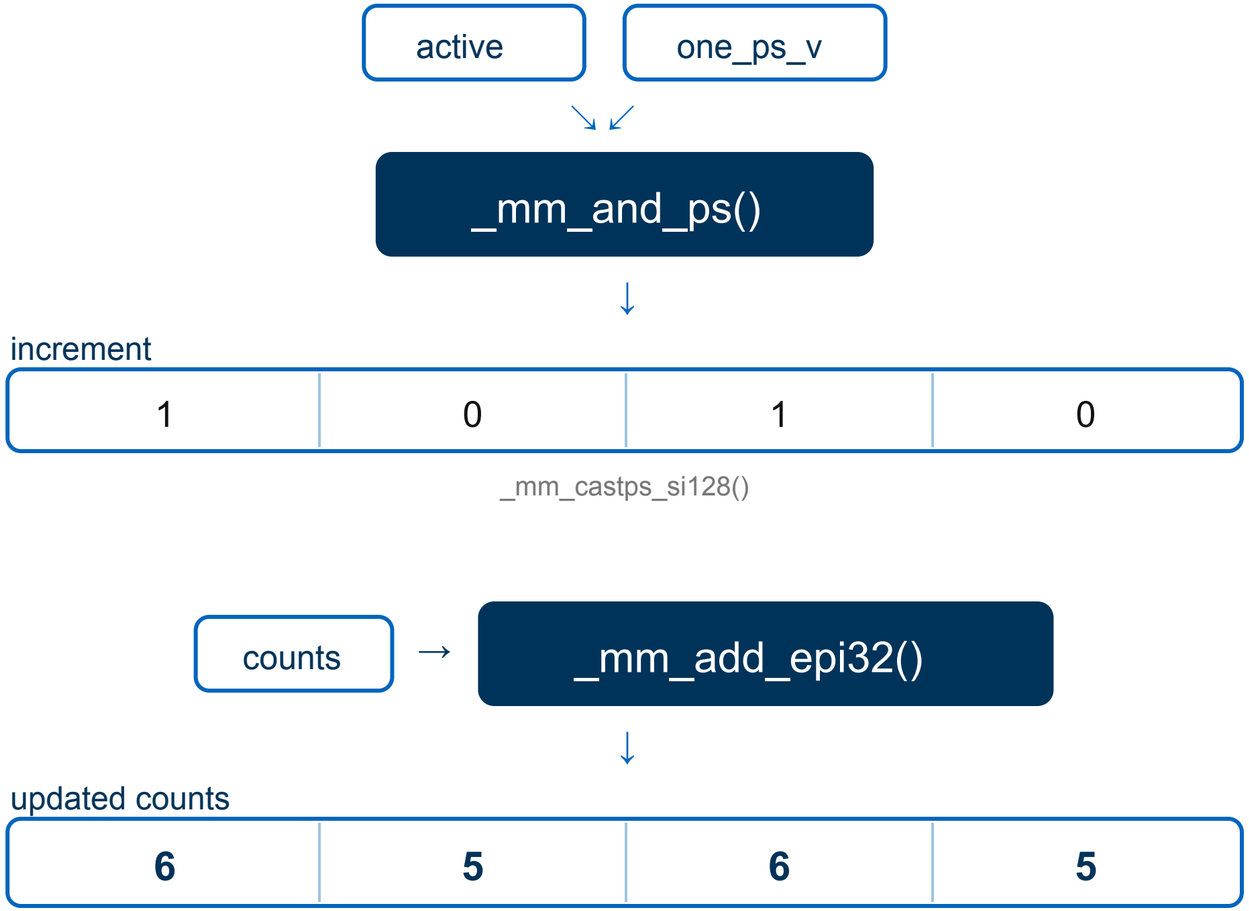
__m128 one_ps_v

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| 0x00000001 | 0x00000001 | 0x00000001 | 0x00000001 |

__m128i counts

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| 5 | 5 | 5 | 5 |

## Dependent operations



# Optimized Counter Update: Subtract the Mask

Because the output of _mm_cmple_ps cast to an int32 is the same as -1 we can directly subtract the counts with this

## The same bits, interpreted as int32

__m128 active

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| 0xFFFFFFFF | 0x00000000 | 0xFFFFFFFF | 0x00000000 |
| active | inactive | active | inactive |

↓

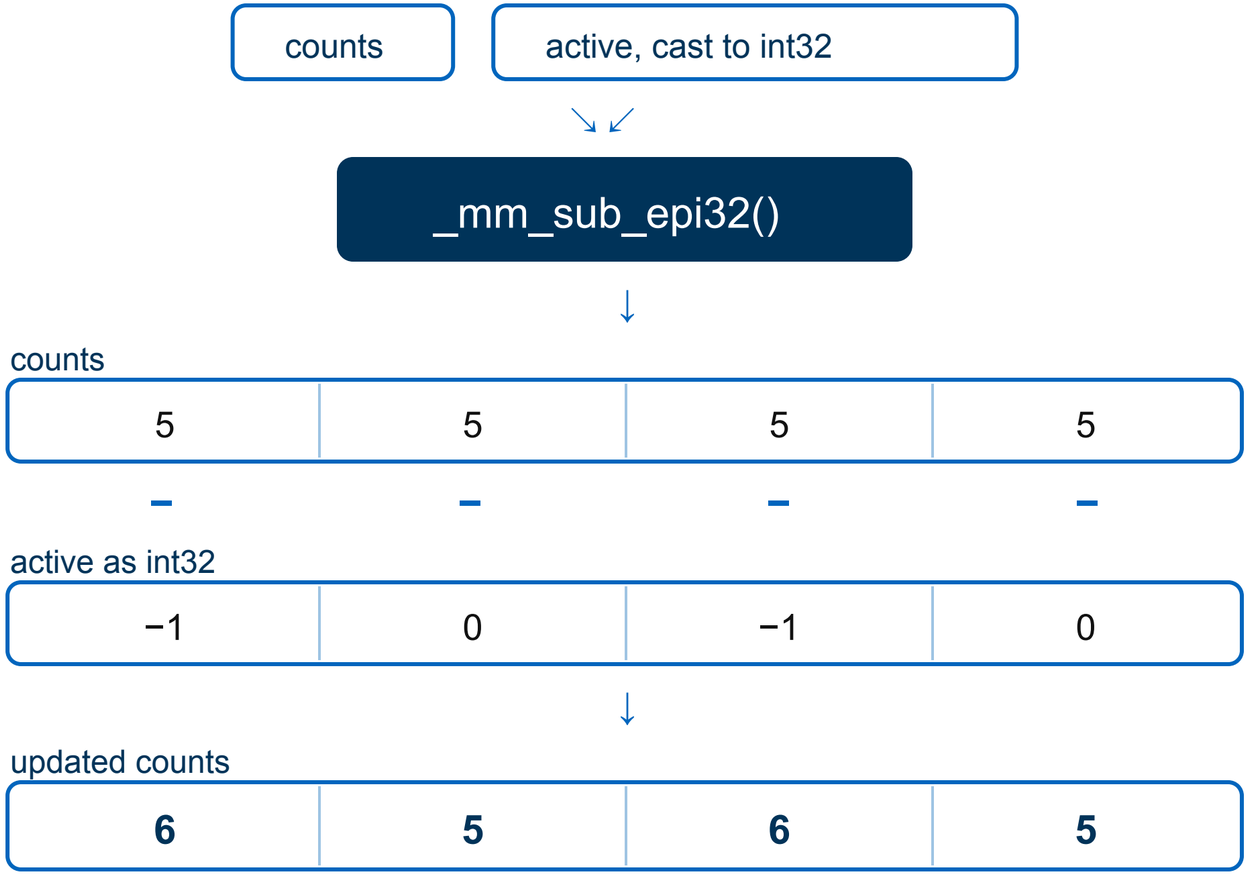
_mm_castps_si128()

↓

__m128i active

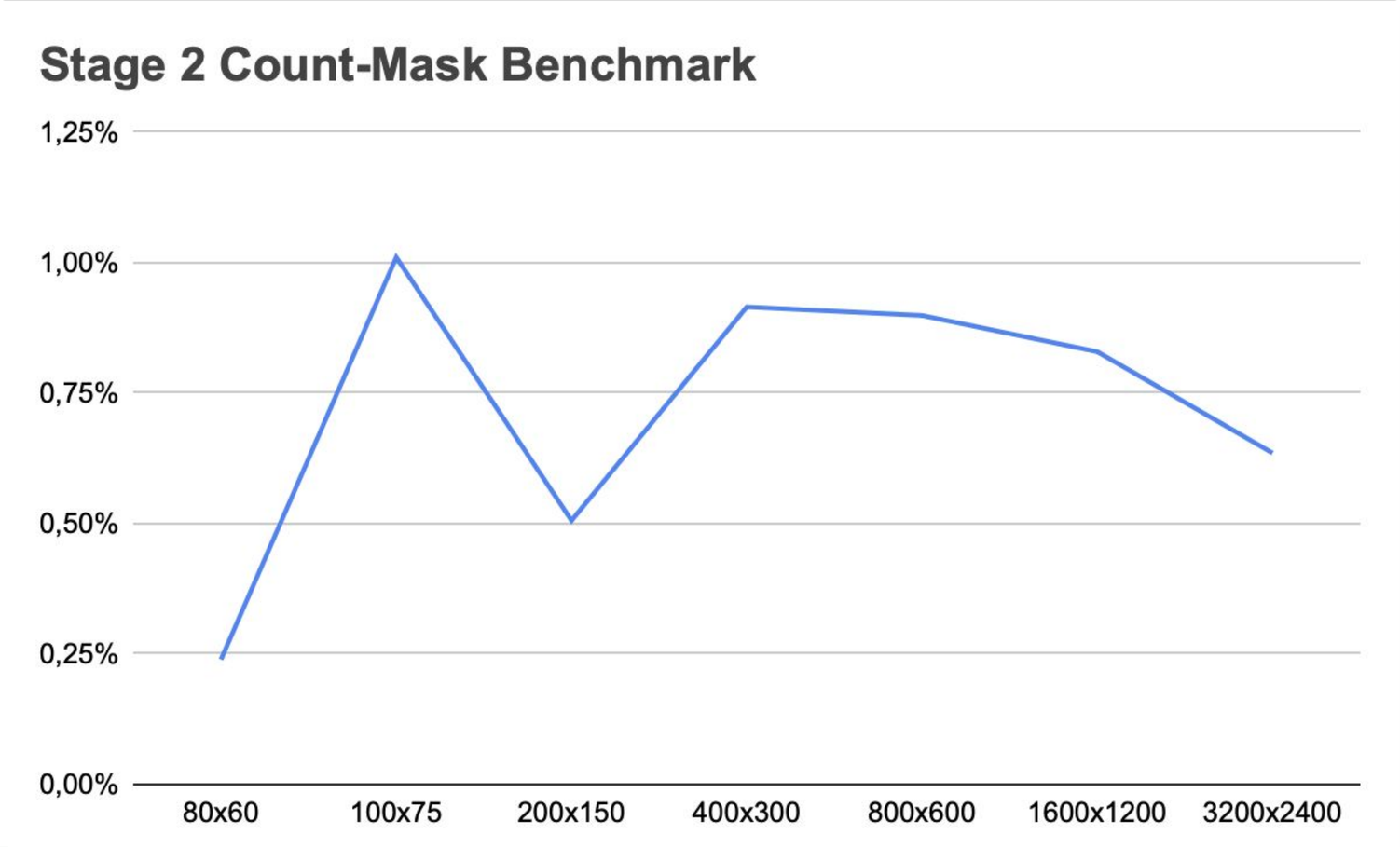
| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| −1 | 0 | −1 | 0 |
| active | inactive | active | inactive |

## Simplified dependency graph



# Optimized Counter Update: Measured Effect

Observed runtime reduction compared with the initial AND-and-ADD counter update.




**0.72%**
mean runtime reduction

**7 / 7**
image sizes faster

**0.24–1.01%**
observed reduction range

Before: AND + ADD After: SUB

**RETAINED** A very small but reproducible and positive measured effect, with one fewer vector operation and one fewer constant register.

# Profiling Exposed the Per-Iteration Exit Chain

A lot of cycles were spent on the iteration exit check.

```
 3.99      cmpleps   %xmm13,%xmm5
          return __builtin_ia32_andps (__A, __B);
21.11       andps     %xmm5,%xmm3
          return __builtin_ia32_movmskps ((__v4sf)__A);
20.25       movmskps  %xmm3,%r9d

          // check to see if any haven't diverged yet
          const __m128 inside = _mm_cmple_ps(magnitude_square
          active = _mm_and_ps(active, inside);  // 0x000000 0

          if (_mm_movemask_ps(active) == 0) { // 0x0000
 0.52       test      %r9d,%r9d
25.01     ↑ jne       258
```

**Required mask update**

**Exit check on the critical path**

movmskps → test → jne

Executed every iteration

**66.37%**

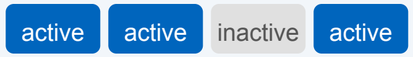
of samples in the active-mask/exit region

52.77% with color output

## Every iteration

inside = _mm_cmple_ps(
    magnitude_squared, four_v);

active = _mm_and_ps(active, inside);




## Optimization candidate

```
if (_mm_movemask_ps(active) == 0)
    break;
```

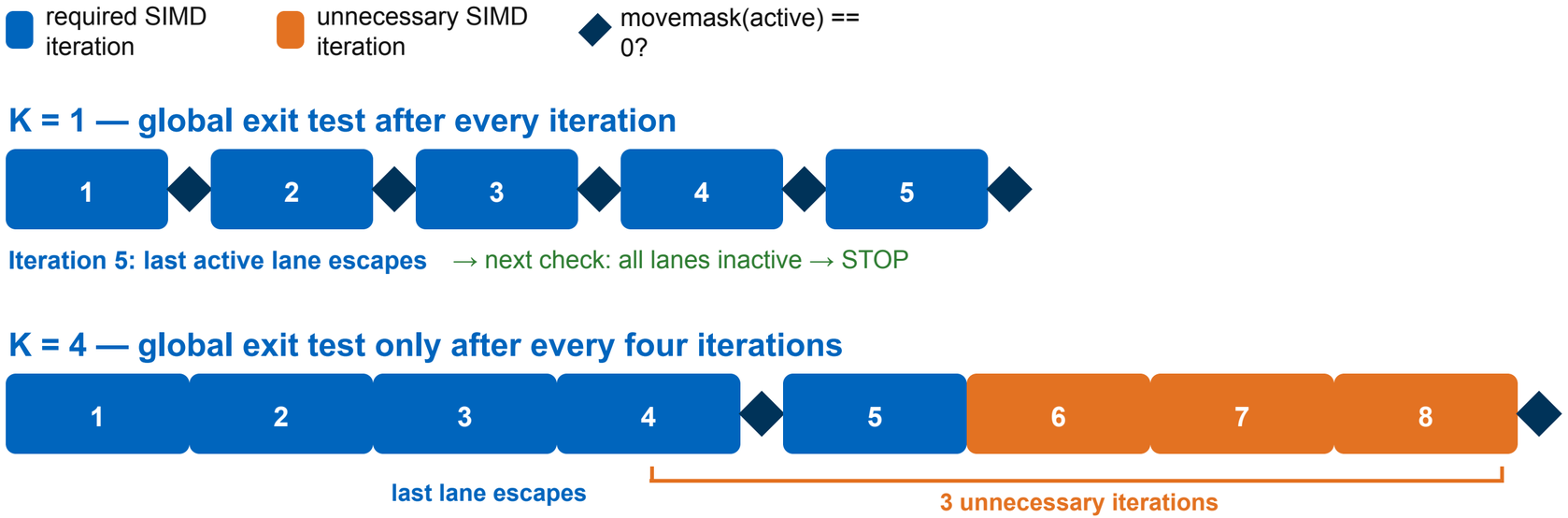
SIMD mask → integer → branch

Every iteration → **Every K iterations**

**Hypothesis: keep the active-mask update, but execute the movemask check less frequently.**

# K-Interval Exit Check: Checking Less Often

K defines how many Julia iterations are executed between the “all lanes inactive?” checks.




The next exit check does not occur until iteration 8.

**Fewer global exit checks ↔ up to K − 1 unnecessary iterations**

**Core hypothesis**

Fewer movemask checks could result in better performance.

K = 1: check every iteration K = 4: check every four iterations

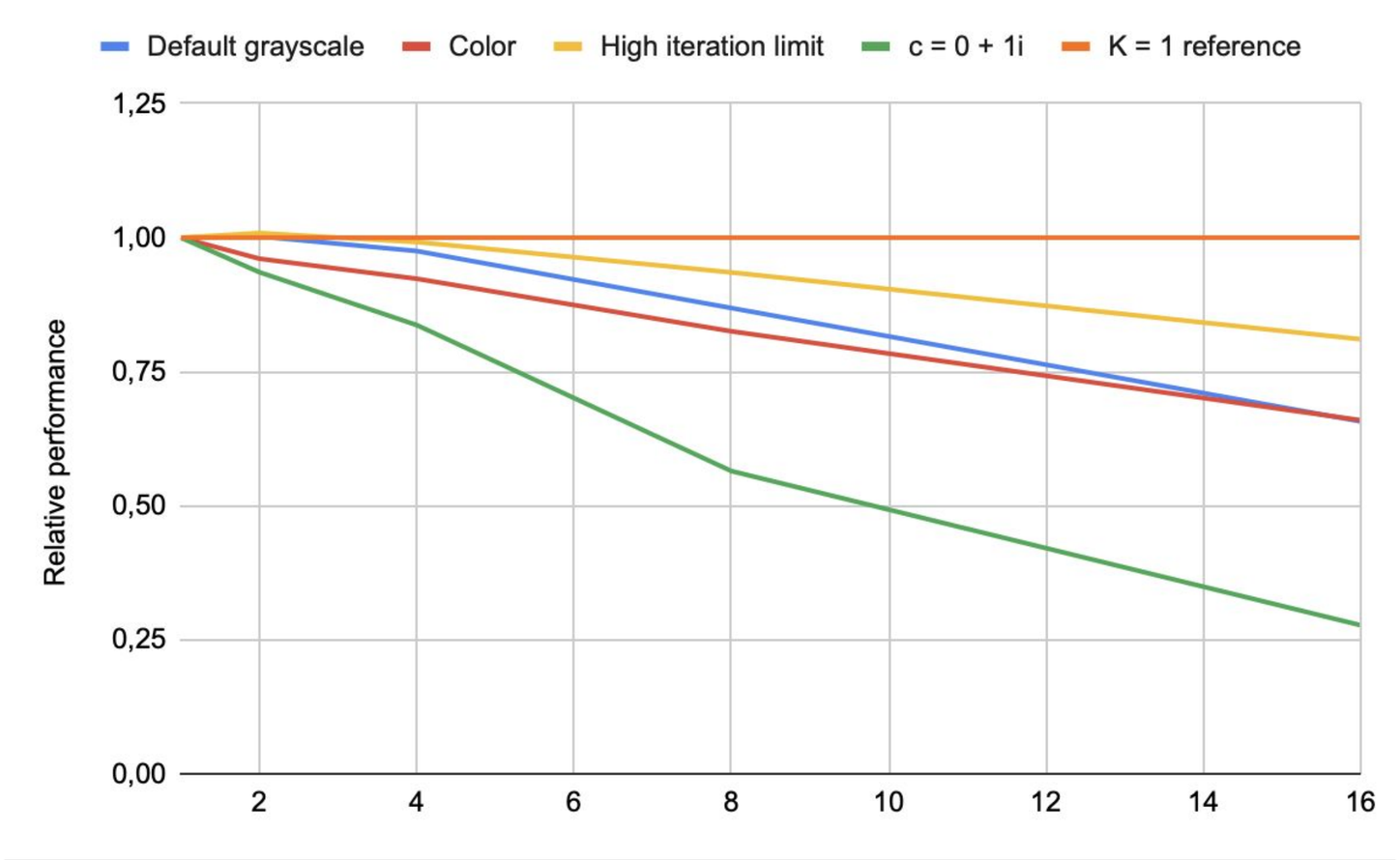
The per-lane escape comparison and active-mask update still execute every iteration. Only the global movemask-and-branch exit is deferred.

**Correctness is preserved:** inactive lanes remain masked and their counters no longer increase. Only loop termination is delayed.

# The K-Sweep Rejected the Optimization

Reducing escape checks produced results within in two workloads but degraded performance in the others.

## K-Interval: Performance Relative to K = 1




**K = 2**
Default grayscale: 1.002x · High iteration: 1.009x

**K = 2**
Color: 0.961x; c = 0 + 1i: 0.936x

**K = 16**
c = 0 + 1i: 0.277×

**No K > 1 improved representative workloads.**

**REJECTED** No consistent gains and workload-dependent slowdowns caused by unnecessary masked iterations didn’t justify keeping this change.

# Go from scalar pixel write to SIMD

## Before: SIMD → array → four scalar conversions

__m128i counts

| Lane 0 | Lane 1 | Lane 2 | Lane 3 |
|---|---|---|---|
| $i_0$ | $i_1$ | $i_2$ | $i_3$ |

↓

_mm_storeu_si128() → unsigned iterations[4]

↓

| iterations[0] ↓ write_pixel(x) | iterations[1] ↓ write_pixel(x+1) | iterations[2] ↓ write_pixel(x+2) | iterations[3] ↓ write_pixel(x+3) |
|---|---|---|---|

```
unsigned iterations[4];
_mm_storeu_si128((__m128i *) iterations, counts);

for (lane = 0; lane < 4; ++lane)
    write_pixel(...);
```

## After: counts stay inside one SIMD register

| **Blue** (counts + 0) & 63 | **Green** (counts + 32) & 63 | **Red** (counts + 16) & 63 |
|---|---|---|

each channel: & 63 → << 2 → 255 - value

↓

blue | (green << 8) | (red << 16)

↓

| Pixel x | Pixel x+1 | Pixel x+2 | Pixel x+3 |
|---|---|---|---|
| B\|G\|R\|0 | B\|G\|R\|0 | B\|G\|R\|0 | B\|G\|R\|0 |

↓

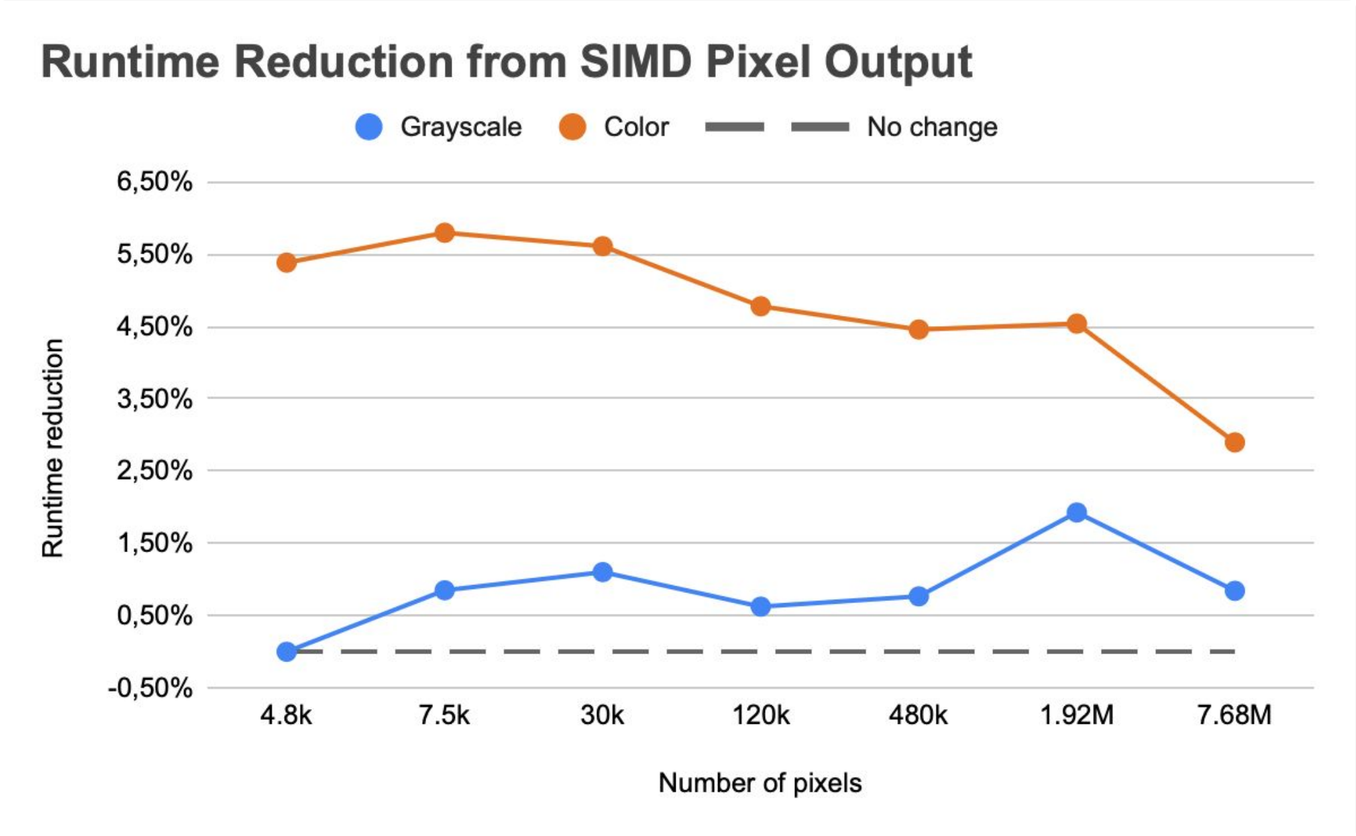
_mm_storeu_si128()

```
write_pixel4(row, x, counts, n, color);
```

**24-bit**
4 pixels = 12 bytes -> shuffle required

**32-bit BGR0**
4 pixels = 16 bytes -> one complete SSE register

# Vectorized Output Primarily Benefits Color Images




**Independent targeted 800 × 600 comparison**

GRAYSCALE

4.354 ms → 4.316 ms **0.87% lower**

Total SIMD speedup: 3.04x → 3.07x

COLOR

4.652 ms → 4.420 ms **5.00% lower**

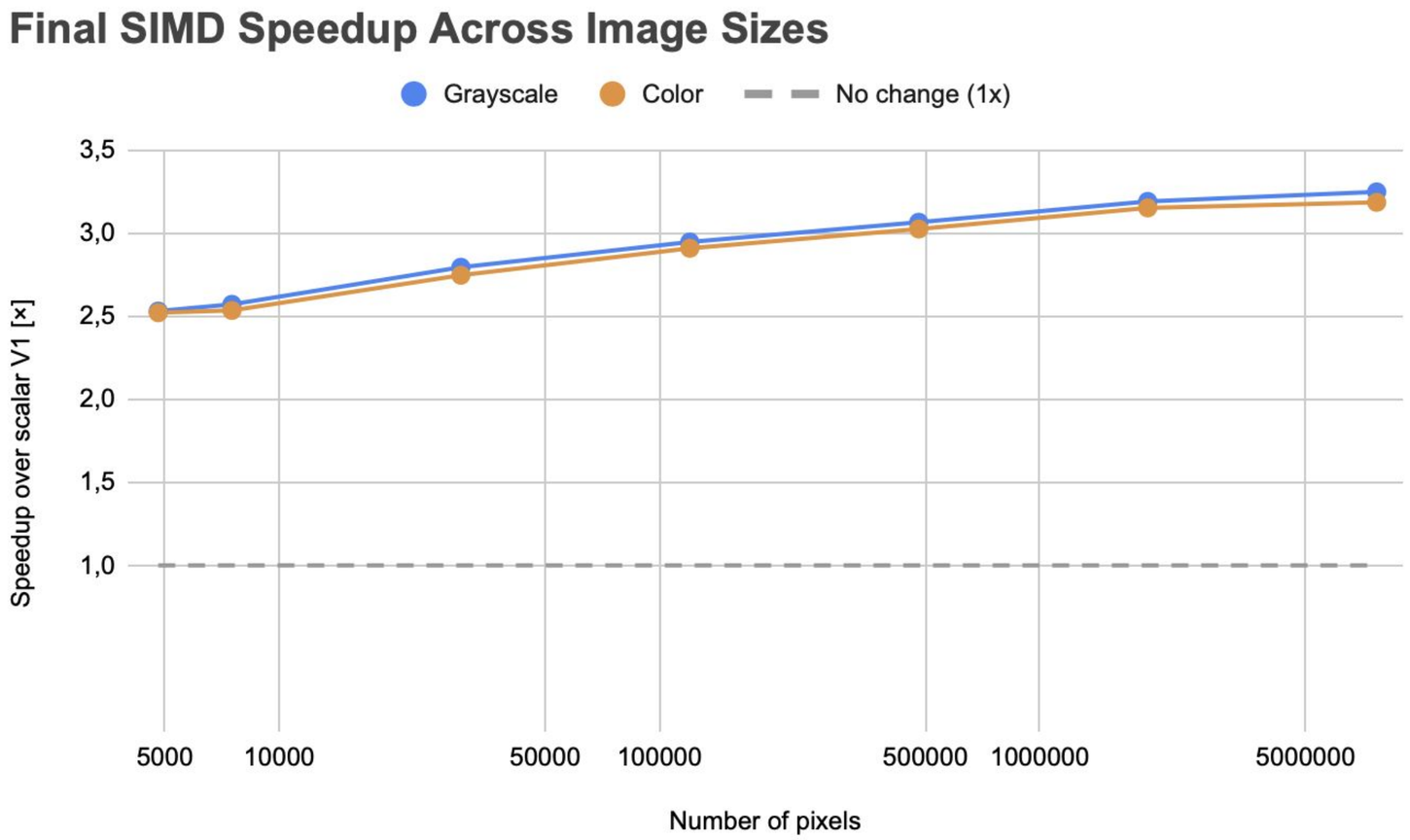
Total SIMD speedup: **2.87x → 3.02x**

**Total SIMD speedup — before vs after vectorized output**


The gap between grayscale and color shrinks from **0.17x** to **0.05x**.

**RETAINED** Color conversion became a significant factor the Julia iteration was vectorized.

# Performance Analysis
# ⇒ Final Speedup Across Image Sizes




Speedup range

**2.52×–3.25×**

Largest grayscale result

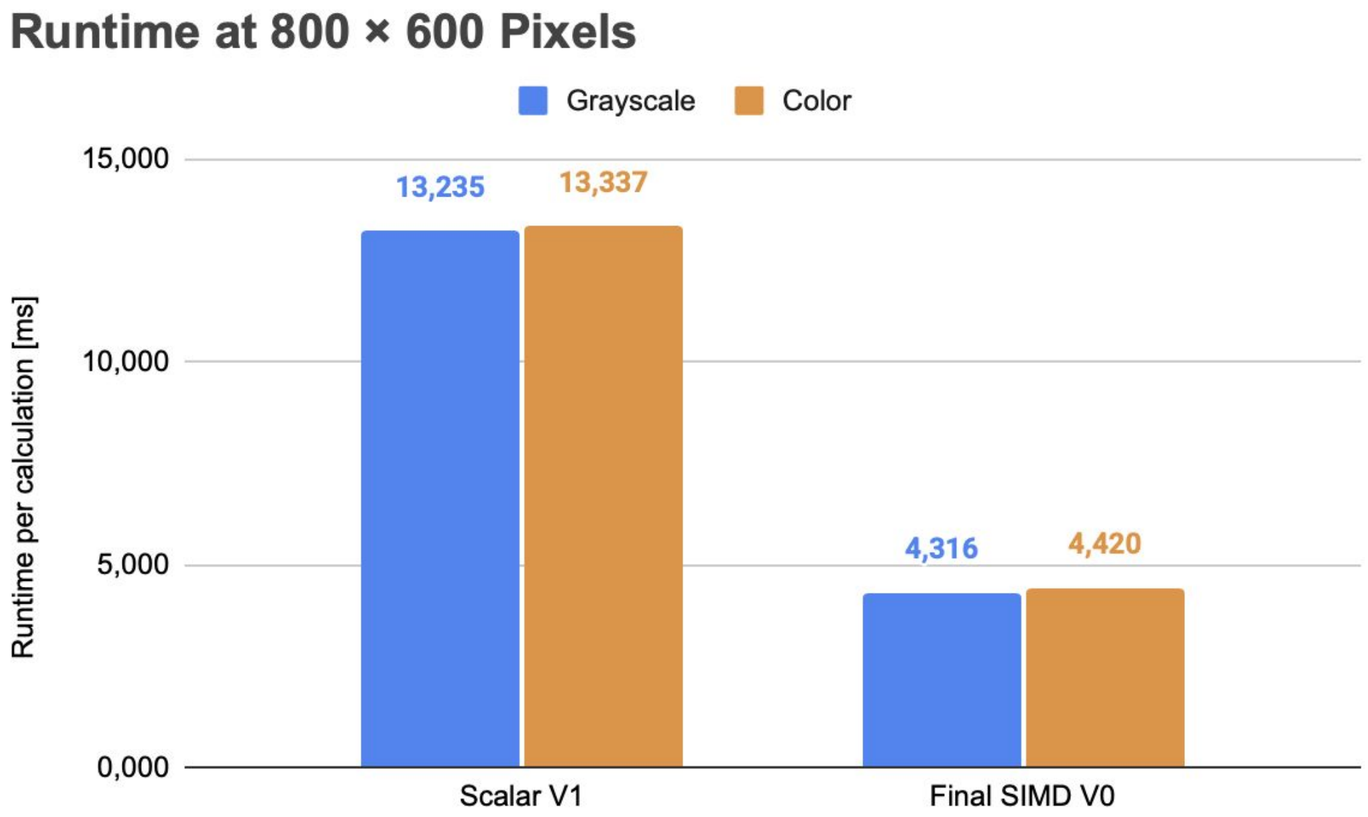
**3.25×**

Largest color result

**3.19×**

Speedup increased with image size as fixed SIMD overhead became less significant.

# ⇒ Runtime at 800 × 600 Pixels




Scalar V1 compared with final SIMD V0

**Grayscale**

**13.235 ms → 4.316 ms**

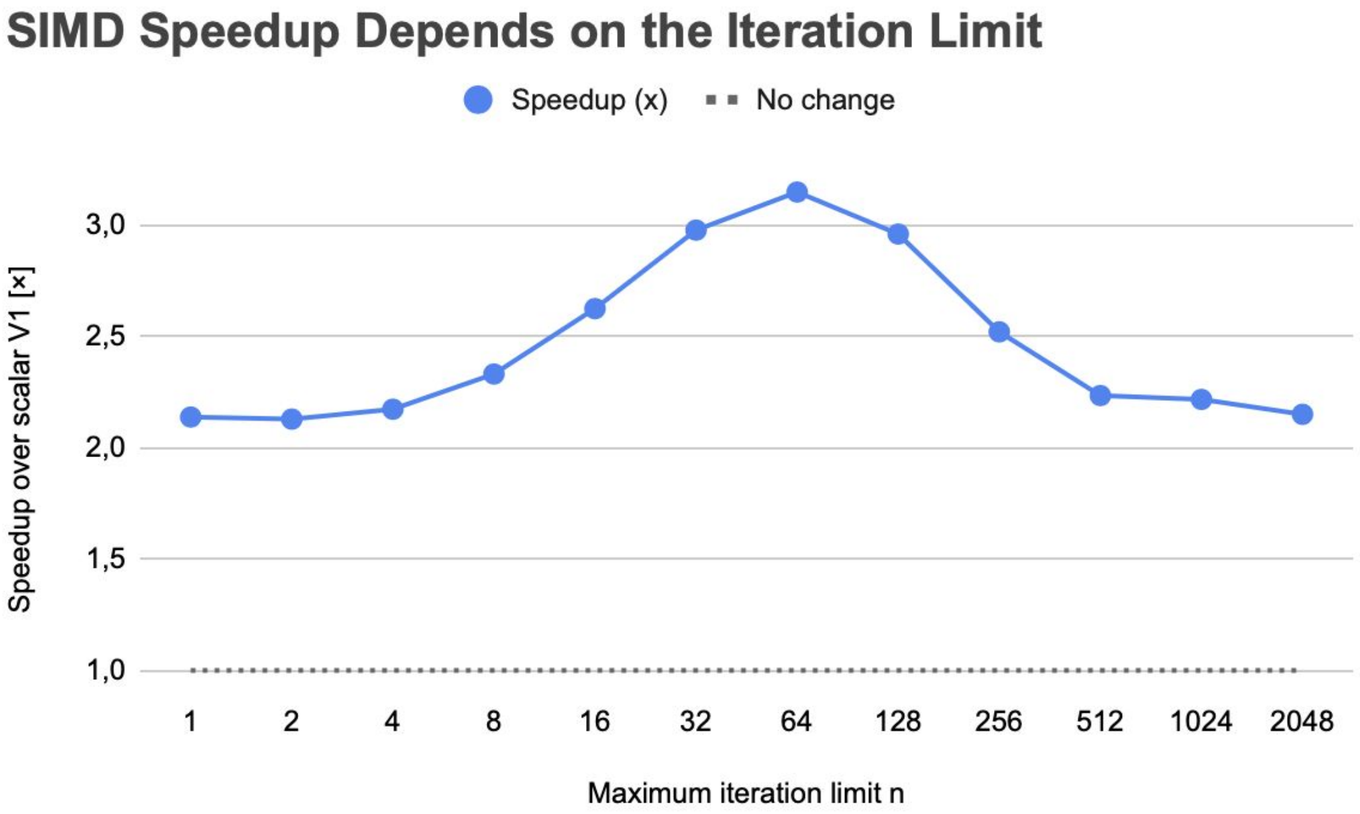
**3.07× speedup**

**Color**

**13.337 ms → 4.420 ms**

**3.02× speedup**

# ⇒ Workload Sensitivity and Outlook




Measured range

**2.13×–3.15×**

Peak result

**3.15× at n = 64**

## Outlook

- Evaluate wider SIMD with AVX2
- Parallelize independent image rows
- Measure repeated independent benchmark batches
- Investigate overhead for very small image widths

# Summary and Outlook

## Key findings

**Vectorisation is the big win.** V0–V2 run 3–4× faster than the scalar baseline V3 across every scenario.

**Add → Sub is a real micro-optimisation.** Counting lanes with _mm_sub_epi32 drops the extra AND, giving V1 a consistent edge over V2.

**V0 ≈ V1.** V0 only adds the every-k mask; with k = 1 the two are effectively identical.

## Why the k-optimisation did not pay off

**Lanes run in lock-step.** All 4 advance until the last escapes, so escaped lanes still do the full mul + add.

**The mask is not the bottleneck.** The per-iteration compare is cheap and branch-predicted, so skipping it saves almost nothing.

**k > 1 adds cost.** Escaped lanes run up to k−1 wasted iterations before detection.

**Outlook:** The arithmetic per iteration is the real limit — wider SIMD (AVX2/AVX-512) or multithreading are the promising next steps.